

DAY 8 — INNER JOIN

Today we start **Week 2: SQL Querying**.

You already learned how to analyze one table. Now we move to one of the most important SQL skills for a Data Engineer:

Combining data from multiple tables using JOINS.

For GenAI/RAG systems, JOINS are everywhere:

users

↓

documents

↓

document_chunks

↓

embeddings

↓

retrieval results

1. What is a JOIN?

Suppose we have two tables.

users

	id	name	country
--	----	------	---------

1	Aarav	India
---	-------	-------

2	Emma	USA
---	------	-----

3	Liam	UK
---	------	----

documents

	id	user_id	title
--	----	---------	-------

101	1	RAG Fundamentals
-----	---	------------------

102	1	PostgreSQL Guide
-----	---	------------------

103	2	Python Basics
-----	---	---------------

104	5	Docker Guide
-----	---	--------------

Notice:

users.id



documents.user_id

The user_id in documents tells us **which user owns the document**.

A JOIN lets us combine these tables.

2. Basic INNER JOIN

Syntax:

```
SELECT
users.name,
documents.title
FROM users
INNER JOIN documents
ON users.id = documents.user_id;
```

Result:

name	title
Aarav	RAG Fundamentals
Aarav	PostgreSQL Guide
Emma	Python Basics

What happened to Liam?

Liam has no matching document.

What happened to document 104?

Its user_id = 5, but user 5 doesn't exist in our users table.

Therefore:

INNER JOIN returns only rows where both tables have a match.

3. The Most Important Concept

Think of:

INNER JOIN

as:

Give me records that exist on BOTH sides of the relationship.

Visually:

users documents

○ ○

\\

\\

\\

MATCHES

Only matching records survive.

4. Understanding ON

This is the most important part:

ON users.id = documents.user_id

It says:

Match the user's id with the document's user_id.

You can think of it as:

users.id

=

documents.user_id

This is usually the relationship between a **primary key** and a **foreign key**.

We'll study primary keys and foreign keys deeply in **Week 3**.

5. Table Aliases

Writing:

users.name

documents.title

works.

But with multiple JOINS, queries become difficult to read.

We can use aliases:

```
SELECT
  u.name,
  d.title
FROM users AS u
INNER JOIN documents AS d
  ON u.id = d.user_id;
```

Now:

u → users

d → documents

This is extremely common in production SQL.

You can also omit AS:

```
FROM users u
INNER JOIN documents d
```

Both are valid.

6. Why Table Qualification Matters

Suppose both tables have an id column.

This is ambiguous:

```
SELECT id
FROM users
INNER JOIN documents
  ON users.id = documents.user_id;
```

Which id?

users.id?

documents.id?

Instead:

```
SELECT
  users.id,
  documents.id
FROM users
```

```
INNER JOIN documents
  ON users.id = documents.user_id;
```

Better:

```
SELECT
  u.id AS user_id,
  d.id AS document_id
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id;
```

This is much clearer.

7. JOIN + WHERE

JOINS become powerful when combined with filtering.

Example:

Find documents belonging to users from India.

```
SELECT
  u.name,
  u.country,
  d.title
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
WHERE u.country = 'India';
```

Logical processing:

users

↓

JOIN documents

↓

matching user-document rows

↓

WHERE country = India

↓

final result

8. JOIN + ORDER BY

Example:

Show documents ordered by page count.

```
SELECT
  u.name,
  d.title,
  d.page_count
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
ORDER BY d.page_count DESC;
```

9. JOIN + LIMIT

Example:

Show the 5 largest documents and their owners.

```
SELECT
  u.name,
  d.title,
  d.page_count
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
ORDER BY d.page_count DESC
LIMIT 5;
```

This pattern appears constantly in APIs.

10. JOIN + DISTINCT

Suppose you want:

Which countries have document owners?

```
SELECT DISTINCT
  u.country
```

```
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
ORDER BY u.country;
```

11. JOIN with Your RAG Database

Imagine:

users

id
name
country
account_status

documents

id
user_id
title
file_type
page_count
status

document_chunks

id
document_id
chunk_text
chunk_index

Relationships:

users

|

| id = user_id

↓

documents

|

| id = document_id

↓

document_chunks

Suppose you want:

Find every chunk and the document title it belongs to.

```
SELECT
d.title,

c.chunk_index,

c.chunk_text

FROM documents d
INNER JOIN document_chunks c
    ON d.id = c.document_id;
```

Now you're starting to query data in a real RAG architecture.

12. Three Important JOIN Questions

Whenever you write a JOIN, ask:

Question 1

What are my tables?

users

documents

Question 2

How are they related?

users.id = documents.user_id

Question 3

Which rows should survive?

For INNER JOIN:

only matching rows

This mental model will save you from many JOIN bugs.

13. INNER JOIN Can Be Written Shorter

These are equivalent:

```
SELECT
u.name,

d.title

FROM users u
INNER JOIN documents d
    ON u.id = d.user_id;
and:
SELECT
u.name,

d.title

FROM users u
JOIN documents d
    ON u.id = d.user_id;
```

In SQL:

```
JOIN
```

usually means:

```
INNER JOIN
```

14. A Very Important Mistake

Don't accidentally join unrelated columns.

For example:

```
ON u.id = d.id
```

This might execute successfully.

But it may be logically wrong.

The correct relationship is:

```
ON u.id = d.user_id
```

SQL checks whether the query is syntactically valid.

It does **not** know your business logic.

This is a major Data Engineering lesson:

A query can be technically valid and still produce incorrect data.

15. JOIN Multiplication

This is extremely important.

Suppose:

User 1

├─ Document A

├─ Document B

└─ Document C

After:

users

JOIN documents

User 1 appears **3 times**.

Result:

user	document
------	----------

Aarav	A
-------	---

Aarav	B
-------	---

Aarav	C
-------	---

This is not duplicate data.

It is the natural result of a **one-to-many relationship**.

1 user

↓

many documents

This concept becomes extremely important when we later combine JOINS with:

COUNT()

SUM()

GROUP BY

16. JOIN + Aggregation Preview

You already know GROUP BY.

Now combine it with JOIN.

Question:

How many documents does each user have?

```
SELECT
  u.id AS user_id,
  u.name,
  COUNT(d.id) AS document_count
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
GROUP BY
  u.id,
  u.name;
```

Result conceptually:

user_id	name	document_count
1	Aarav	2
2	Emma	2
3	Liam	1
4	Sofia	2
6	Olivia	2

Notice something interesting:

Users without documents disappear because we're using:

INNER JOIN

We'll solve that problem tomorrow with:

Day 9 — LEFT / RIGHT / FULL JOIN

17. Production RAG Example

Imagine your application stores:

users

documents

document_chunks

A user asks:

"Show me all chunks from my processed documents."

SQL:

```
SELECT
d.id AS document_id,
d.title,
c.chunk_index,
c.chunk_text
FROM documents d
INNER JOIN document_chunks c
  ON d.id = c.document_id
WHERE d.status = 'processed';
```

This is already close to the kind of SQL you'll encounter in a real RAG backend.

18. Debugging Challenge #1

This query:

```
SELECT
u.name,
d.title
FROM users u
INNER JOIN documents d
  ON u.id = d.id;
```

It runs.

But suppose the relationship is:

users.id

documents.user_id

Your task

Explain:

1. Why is the query wrong?
2. What should the ON condition be?
3. Why can a wrong JOIN still execute successfully?

Don't look for an answer from me. Write your explanation.

19. Debugging Challenge #2

Query:

```
SELECT
  u.name,
  d.title
FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
WHERE d.page_count > 40;
```

A developer says:

"I only wanted documents over 40 pages, so why are users involved?"

Explain why the JOIN is still useful.

20. SQL Exercises — YOU WRITE THE SQL

Use the Day 7 tables:

users

id | name | email | country | account_status

documents

id | user_id | title | file_type | page_count | status

Basic

Q1

Return:

user name

document title

for every document.

Q2

Return:

user_id

user_name

document_id

document_title

Use clear aliases.

Q3

Find all documents belonging to users from:

India

Return:

user_name

title

country

Q4

Find documents owned by users whose account status is:

active

Return:

user_name

title

account_status

Intermediate

Q5

Return all documents with:

page_count > 30

and include the owner's name.

Q6

Return the 5 largest documents, including:

document title

owner name

page count

Q7

Return unique countries that have at least one document.

Q8

Find all processed documents belonging to users from India.

Return:

user_name

document_title

file_type

page_count

status

Q9

Find all PDFs owned by active users.

Return:

user_name

document_title

page_count

Order by:

page_count DESC

21. Engineering Challenge

Imagine your RAG application has:

users

documents

The product manager asks:

"Give me the top 3 users who uploaded the largest individual documents."

Important:

They are asking for **documents**, not total pages per user.

Your result should contain:

user_name

document_title

page_count

Sort from largest document to smallest.

Requirement

Use:

INNER JOIN

ORDER BY

LIMIT

Do it yourself.

22. Interview Question

Answer this in your own words:

What is the difference between INNER JOIN and WHERE?

Don't just say:

"JOIN combines tables."

Explain their different responsibilities.

23. Explanation Challenge

Explain this query line by line:

```
SELECT
u.name,

d.title,

d.page_count

FROM users u
INNER JOIN documents d
  ON u.id = d.user_id
WHERE d.status = 'processed'
ORDER BY d.page_count DESC
LIMIT 3;
```

Your explanation must cover:

1. What u represents
2. What d represents
3. What ON does
4. What WHERE does
5. Why ORDER BY is used
6. Why LIMIT is used
7. What the final result represents

24. Data Engineering Thinking

Imagine you have:

users → 10 million rows

documents → 500 million rows

and run:

```
SELECT
u.name,

d.title

FROM users u
INNER JOIN documents d
  ON u.id = d.user_id;
```

Think about:

- Which table is larger?
- Which column is used for matching?
- Could an index help?
- How many rows could the JOIN produce?
- What happens if one user has 100,000 documents?
- Why could the result become enormous?

You don't need to know query optimization yet.

Just start thinking like a Data Engineer.

25. Short Test

Don't use Google. Don't ask me for answers before attempting.

Q1

What does INNER JOIN return?

- A. All rows from the first table
- B. All rows from both tables
- C. Only matching rows
- D. Only unmatched rows

Q2

Given:

users.id

documents.user_id

what should normally connect them?

A.

ON users.id = documents.id

B.

ON users.id = documents.user_id

C.

ON users.user_id = documents.id

D.

ON users.name = documents.title

Q3

Why do we use:

u.name

instead of simply:

name

when querying multiple tables?

Q4

Write SQL to return:

user name

document title

for documents with:

page_count > 50

Q5

Explain why a user can appear multiple times after an INNER JOIN with documents.